

Mobile Camera Intrinsic Calibration Report

LAB0-1: Camera Calibration Experiment

Name: Boyang Xie
Student ID: 24020036030
Course/Class: F29XS

Data status: An OpenCV calibration was completed using 18 real JPEG images from an iPhone 15 Pro rear camera. The checkerboard has 8×5 inner corners and 3 cm squares.

1 Objectives

This experiment uses Python and OpenCV to perform camera calibration for a mobile-camera image set captured with a planar checkerboard. Its objectives are:

1. to understand the relationships among the world, camera, image, and pixel coordinate systems and the pinhole camera model;
2. to distinguish intrinsic parameters from extrinsic parameters and understand the roles of focal length, principal point, and lens distortion coefficients;
3. to use checkerboard corner detection, subpixel refinement, and `cv2.calibrateCamera`;
4. to obtain the camera matrix, distortion coefficients, and per-view poses, and to evaluate the fit using reprojection error;
5. to assess how checkerboard coverage, camera pose, and corner quality affect the calibration result.

All numerical results in this report come from the recorded JPEG data and the official NPZ result file. No hardware or calibration value is inferred from a default configuration.

2 Principles

2.1 Coordinate Systems and the Pinhole Model

The world coordinate system describes points on the calibration board. The camera coordinate system is centered at the optical center. The image coordinate system represents continuous image-plane coordinates, whereas the pixel coordinate system uses discrete image coordinates referenced to the upper-left image corner. Let $\mathbf{X}_w = (X_w, Y_w, Z_w)^\top$ be a point in world coordinates and $\mathbf{X}_c = (X_c, Y_c, Z_c)^\top$ the same point in camera coordinates. The extrinsic parameters are a rotation matrix $\mathbf{R}$ and a translation vector $\mathbf{t}$:

$$\mathbf{X}_c = \mathbf{R}\mathbf{X}_w + \mathbf{t}. \quad (1)$$

Under the pinhole model, let $x = X_c/Z_c$ and $y = Y_c/Z_c$ be normalized image coordinates. The projection to pixel coordinates is

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{R} \quad \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \quad s \neq 0, \quad (2)$$

where (u, v) are pixel coordinates and $\mathbf{K}$ is the camera matrix:

$$\mathbf{K} = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \quad (3)$$

Here f_x and f_y are focal lengths in pixels, (c_x, c_y) is the principal point, and γ is the skew parameter. For ordinary mobile-camera images, γ is commonly close to zero.

2.2 Lens Distortion Model

Real lenses make the observed image deviate from ideal pinhole projection. Let $r^2 = x^2 + y^2$. A common radial distortion model is

$$\begin{aligned} x_r &= x (1 + k_1 r^2 + k_2 r^4 + k_3 r^6), \\ y_r &= y (1 + k_1 r^2 + k_2 r^4 + k_3 r^6). \end{aligned} \quad (4)$$

The radial distortion coefficients are k_1, k_2, k_3 . Tangential distortion can be written as

$$\begin{aligned} x_d &= x_r + 2p_1 xy + p_2 (r^2 + 2x^2), \\ y_d &= y_r + p_1 (r^2 + 2y^2) + 2p_2 xy. \end{aligned} \quad (5)$$

The final pixel coordinates are

$$u = f_x x_d + \gamma y_d + c_x, \quad v = f_y y_d + c_y. \quad (6)$$

OpenCV commonly returns five coefficients in the order $(k_1, k_2, p_1, p_2, k_3)$. If an interface returns a different length, the report must preserve the returned order and length.

2.3 Planar Calibration and the Zhang Method

Set the checkerboard plane to $Z_w = 0$ and define board coordinates using the square size. The mapping from a point on the plane to its image is a homography:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X_w \\ Y_w \\ 1 \end{bmatrix}, \quad \mathbf{H} = \mathbf{K} [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{t}], \quad (7)$$

where $\mathbf{r}_1$ and $\mathbf{r}_2$ are the first two columns of the rotation matrix. Multiple views of the same checkerboard provide homography constraints. The orthogonality and unit-length constraints of the rotation columns jointly estimate the intrinsic parameters and then the pose of each view. A nonlinear optimization finally refines the intrinsic parameters, distortion coefficients, and extrinsic parameters by minimizing the discrepancy between observed and projected corners.

In the program, the checkerboard object points lie on $Z_w = 0$, and the detected image points form the corresponding two-dimensional observations. The function `cv2.calibrateCamera` performs the calibration from these correspondences. The checkerboard configuration and square size must match the physical board; otherwise the translation-vector scale is not physically meaningful.

2.4 Reprojection Error

Let $\mathbf{q}_{j,i}$ be the observed pixel for corner i in image j , and let $\hat{\mathbf{q}}_{j,i}$ be the pixel obtained by projecting the estimated three-dimensional point with `cv2.projectPoints`. The pointwise reprojection error is

$$e_{j,i} = \|\hat{\mathbf{q}}_{j,i} - \mathbf{q}_{j,i}\|_2. \quad (8)$$

If M is the total number of valid corners, the mean pointwise reprojection error and global RMS are

$$\bar{e} = \frac{1}{M} \sum_{j,i} e_{j,i}, \quad e_{\text{RMS}} = \sqrt{\frac{1}{M} \sum_{j,i} e_{j,i}^2}. \quad (9)$$

For image j , the per-image RMS and its arithmetic mean are

$$e_{j,\text{RMS}} = \sqrt{\frac{1}{M_j} \sum_i e_{j,i}^2}, \quad \bar{e}_{\text{view}} = \frac{1}{n} \sum_{j=1}^n e_{j,\text{RMS}}, \quad (10)$$

where M_j is the number of corners in image j and n is the number of valid images. The NPZ fields `rms_error`, `mean_reprojection_error`, and `mean_per_image_rms` record these three distinct quantities.

Table 1: Experimental environment and acquisition record

Category	Item	Recorded value
Operating system	System and architecture	macOS 26.6.2, Darwin 25.6.0, arm64
Software	Python	3.13.12
Software	OpenCV	4.10.0 (cv2)
Software	Main APIs	<code>findChessboardCorners</code> , <code>cornerSubPix</code>
Software	Main APIs	<code>calibrateCamera</code> , <code>projectPoints</code>
Hardware	Phone model	Apple iPhone 15 Pro (JPEG EXIF)
Hardware	Camera mode	Rear camera; focal length not recorded
Image	Image resolution	5712×4284
Board	Inner-corner configuration	8×5
Board	Physical square size	3 cm
Data	Input images	18 JPEG files (<code>task2/images_jpg/</code>)
Data	Valid/failed images	18 / 0

3 Experimental Environment

The software environment was verified directly. The Python interpreter was `/Users/jiaqiaosu/miniconda3/bin/python`; it imports `cv2`. The JPEG EXIF records an Apple iPhone 15 Pro, and the data were captured with its rear camera. The specific rear-lens focal length was not recorded and is not inferred here.

The software versions, JPEG device information, image size, checkerboard configuration, 3 cm square size, and image counts were verified from local commands, EXIF, acquisition records, and `camera_params.npz`. The program was run with `square_size=3.0`, so the translation vectors are measured in cm. The specific rear-lens focal length was not recorded.

4 Data Acquisition

4.1 Checkerboard and Coordinate Convention

The calibration target is a planar black-and-white checkerboard. The option `--checkerboard` specifies the number of checkerboard inner-corner columns and rows, while `--square-size` specifies the physical side length of one square. The present run used 8×5 inner corners and a 3 cm square size, with `square_size=3.0`. The upper-left inner corner was set to $(0, 0, 0)$; the two board directions define X_w and Y_w , and all board points have $Z_w = 0$. This convention is consistent with the planar model in Equation 7.

4.2 Acquisition Conditions

The input set contains 18 JPEG files under `task2/images_jpg/`, each with resolution 5712×4284 . JPEG EXIF records an Apple iPhone 15 Pro, and the acquisition used its rear camera. The specific rear-lens focal length was not recorded and is not inferred. The original HEIC files under `task2/images/` were not used as direct OpenCV input because the current `cv2.imread` cannot read them.

The board should be captured at different distances, translations, and tilt angles so that it covers the image center, edges, and different depths. A set consisting only of nearly frontal views with repeated poses is insufficient. The board should be complete and sharp, without severe reflection, motion blur, occlusion, or overexposure. A separate pose-coverage log remains to be organized from the original acquisition.

The program reads `task2/images_jpg/` and preserves the original filenames, allowing the NPZ `image_paths` field to be matched to each detection result.

The calibration values come from the actual run on the 18 JPEG files. The rear-camera mode, checkerboard configuration, and 3 cm square size are fixed acquisition facts; only the exact rear-lens focal length and the detailed pose-coverage log remain undocumented.

Table 2: Mobile-camera acquisition record

Item	Recorded content	Status
Phone	Manufacturer and model	Apple iPhone 15 Pro (JPEG EXIF)
Camera	Front/rear camera and lens mode	Rear camera; focal length not recorded
Image resolution	Width $\times$ height (pixels)	5712 $\times$ 4284
Image count	Input JPEG files	18
Checkerboard	Inner corners per row/column	8 $\times$ 5
Square size	Physical side length	3 cm; <code>square_size=3.0</code>
Pose coverage	Position, distance, and tilt	Acquisition log to be organized

5 Experimental Procedure

1. Prepare the planar checkerboard, measure the physical square size, and record the number of inner corners in each direction.
2. Capture clear images at different distances, translations, and tilt angles. Record the phone model, rear-camera use, image resolution, and acquisition settings.
3. Read each image and check whether it can be loaded. A read failure is recorded for that file and does not stop processing of the remaining images.
4. Convert a color image to grayscale with `cv2.cvtColor`, then detect checkerboard inner corners with `cv2.findChessboardCorners`. Record the two-dimensional corners for successful images and the filename and reason for failures.
5. Refine detected corners to subpixel accuracy with `cv2.cornerSubPix`.
6. Generate planar three-dimensional object points from the checkerboard configuration and square size, and pair them with the two-dimensional image points. The geometry follows Equation 7.
7. Check the number of valid images and the consistency of image sizes. Mixed resolutions and fewer valid views than `--min-images` must be rejected.
8. Call `cv2.calibrateCamera` to estimate the camera matrix, distortion coefficients, and the rotation and translation vectors for each valid view. The camera matrix is defined in Equation 3; the distortion model is given in Equations 4 and 5.
9. Reproject the three-dimensional points with `cv2.projectPoints`, compute pointwise errors using Equation 8, and summarize the three error definitions in Section 7.
10. Save the camera matrix, distortion coefficients, poses, error statistics, image counts, and detection status to the NPZ result. If requested, use `cv2.undistort` to save corrected images.
11. Use the recorded result and the criteria in Section 8 to assess parameter consistency and data quality.

The complete control flow implemented by the OpenCV program is summarized in Figure 1.

6 Program Design

6.1 Input, Configuration, and Output

The Python entry point uses `argparse` and reads JPEG files from `task2/images_jpg/`.

The options `--checkerboard` and `--square-size` specify the checkerboard inner-corner configuration and square size. The options `--images` and `--output-dir` specify the input pattern and output directory. The option `--min-images` sets the minimum number of valid views.

The present run used 8 $\times$ 5 inner corners, `square_size=3.0` cm, and `--min-images 3`. Each image was read, converted to grayscale, searched for corners, and refined to subpixel accuracy. Successful files print `[OK]`; read failures, corner failures, and resolution mismatches print `[FAILED]` with a reason and are counted. All 18 JPEG files succeeded in this run. HEIC files under `task2/images/` are not directly readable by the current `cv2.imread`.

6.2 Core Computation

Three-dimensional board points are generated from the checkerboard geometry, while two-dimensional points are obtained from the refined corners. Calling `cv2.calibrateCamera` produces the camera matrix, distortion coefficients, and a rotation and translation vector for each valid view. The call receives `objectPoints`,

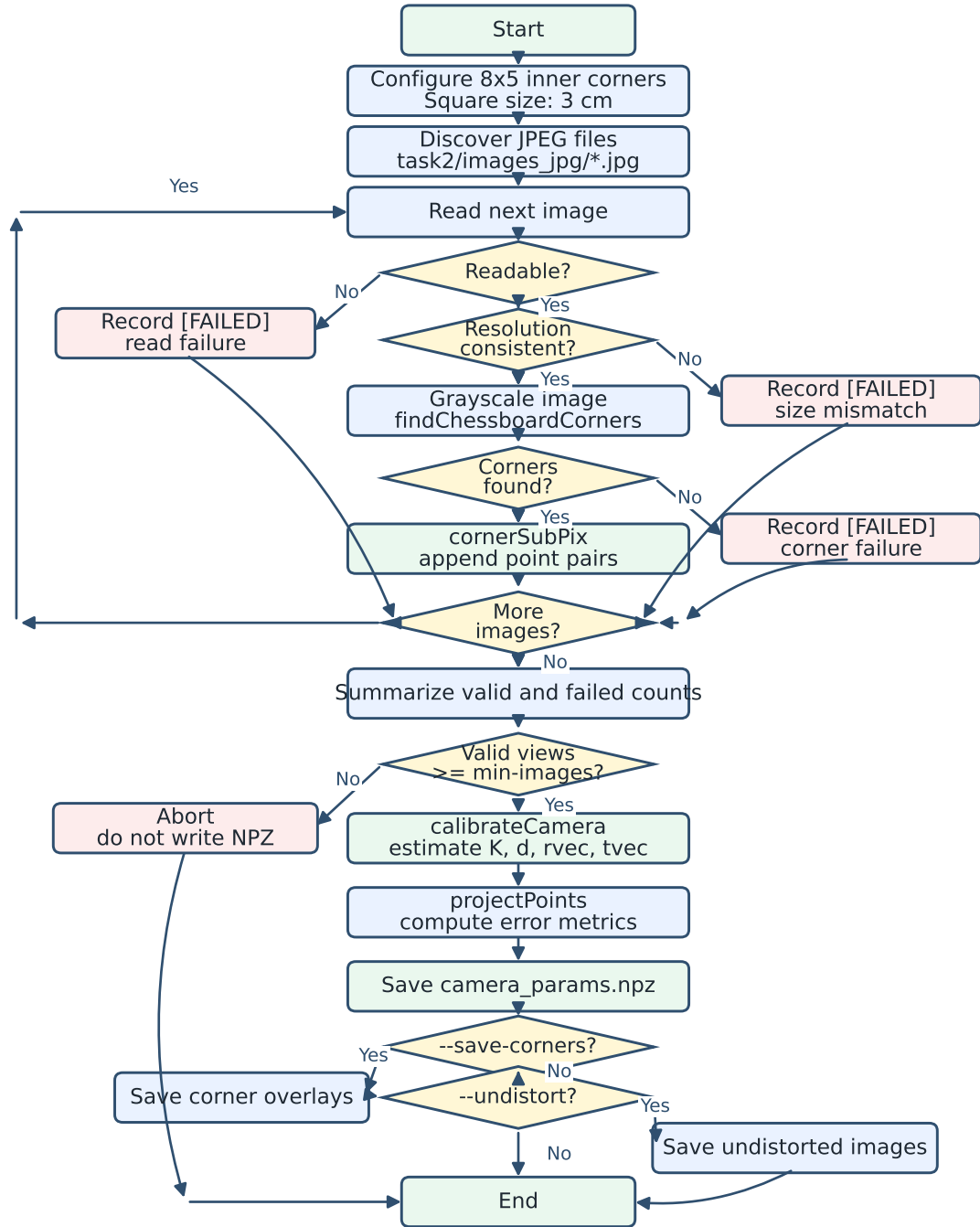


Figure 1: OpenCV camera-calibration workflow implemented in this experiment.

`imagePoints`, and `imageSize`; the projection relation is given in Equation 2.

The program uses `cv2.projectPoints` to reproject every board point and compare it with the detected pixel. It stores OpenCV RMS in `rms_error`, the per-image RMS array in `per_image_reprojection_error`, the global pointwise mean in `mean_reprojection_error`, and the arithmetic mean of per-image RMS values in `mean_per_image_rms`. Their definitions are given in Equations 9 and 10.

The official result is written to `task2/output/camera_params.npz`. The file records the image size, checkerboard configuration, poses, candidate/valid/failed paths, failure reasons, and all three error statistics. The 3.0 square-size value is in cm, so the translation vectors are in cm. Hardware mode and physical board measurements are acquisition facts, not values inferred by the program.

7 Experimental Results

The results below were read directly from `task2/output/camera_params.npz` for the 18 real JPEG files in `task2/images_jpg/*.jpg`. The run used an 8×5 checkerboard, a 3.0 cm square size, and `--min-images 3`. These values are recorded outputs, not estimates or placeholders.

$$\mathbf{K} = \begin{bmatrix} 4082.15290893 & 0 & 2883.66417823 \\ 0 & 4083.58657686 & 2107.20293734 \\ 0 & 0 & 1 \end{bmatrix}. \quad (11)$$

$$\mathbf{d} = [0.20245123, -0.29420837, -0.00326679, 0.00308671, -1.04554325], \quad (12)$$

where $\mathbf{d}$ follows the common OpenCV order $(k_1, k_2, p_1, p_2, k_3)$.

Table 3: Mobile-camera calibration results read from the official NPZ file

Category	Metric	Recorded value
Data	Input pattern	<code>task2/images_jpg/*.jpg</code>
Data	Candidate images	18
Data	Valid calibration images	18
Data	Failed images	0 (read, corner, and size failures)
Data	Image size (width $\times$ height)	5712×4284
Configuration	Checkerboard inner corners	8×5
Configuration	<code>square_size</code>	3.0 cm
Intrinsics	Camera matrix $\mathbf{K}$	See Equation 11
Intrinsics	f_x	4082.15290893
Intrinsics	f_y	4083.58657686
Intrinsics	c_x	2883.66417823
Intrinsics	c_y	2107.20293734
Distortion	$(k_1, k_2, p_1, p_2, k_3)$	See Equation 12
Error	OpenCV RMS (pixels)	2.06424401
Error	Mean pointwise reprojection error (pixels)	1.65696931
Error	Mean per-image RMS (pixels)	1.90750642
Extrinsics	Rotation/translation shape	$18 \times 3 \times 1$; translations in cm

OpenCV RMS is the global root-mean-square error returned by `cv2.calibrateCamera`. The mean pointwise reprojection error is `mean_reprojection_error`, averaged over all $18 \times 40 = 720$ valid corners. The mean per-image RMS is `mean_per_image_rms`, the arithmetic mean of 18 per-image RMS values. These definitions are distinct. The per-image array is stored in `per_image_reprojection_error`; poses are stored in `rotation_vectors` and `translation_vectors`.

Figure 2 shows one of the 18 valid views rather than repeating all outputs. The left image checks coverage of the 8×5 inner corners, while the right image illustrates the undistortion result. Both were generated with `--save-corners --undistort`.

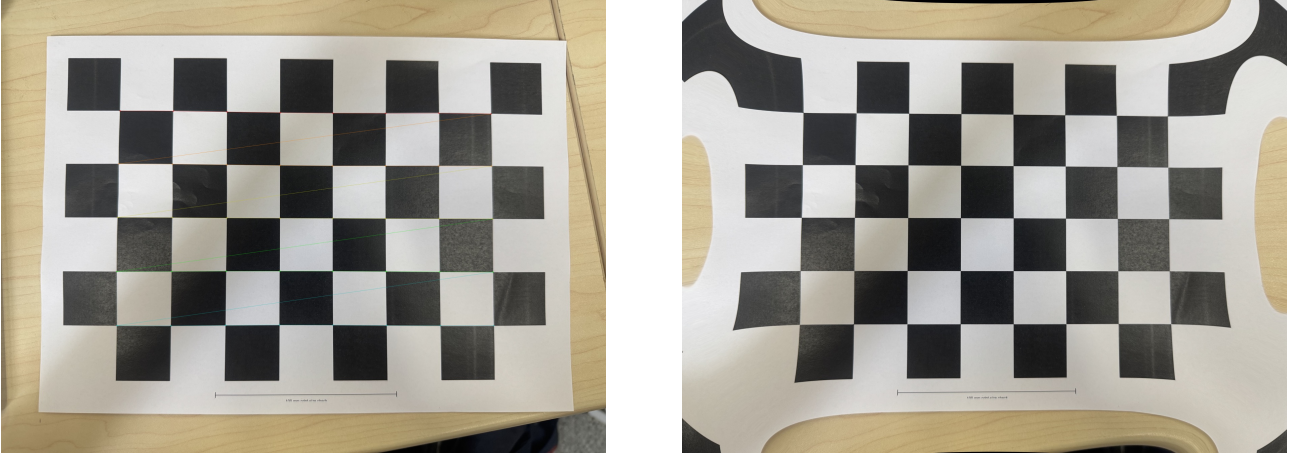


Figure 2: Representative results for `IMG_5486.jpg`: the left image shows the detected corners, and the right image is the undistorted image produced with the same calibration. The figures come from the temporary visualization run; numerical results remain those in `task2/output/camera_params.npz`.

8 Analysis

This analysis is based on 18 real JPEG images, the official result file, and the corresponding run. Conclusions are limited to the current 5712×4284 images, 8×5 checkerboard inner corners, and 3 cm square size; a pixel error is not treated as a direct guarantee of physical measurement accuracy.

8.1 Intrinsic-Parameter Consistency

The estimated f_x , f_y , c_x , and c_y are finite and positive. From the measured focal lengths,

$$\rho_f = \frac{|f_x - f_y|}{(f_x + f_y)/2} = 0.0003511422 \quad (0.035114\%).$$

The two focal-length estimates are therefore very close, which is compatible with nearly square pixel scaling in this image set. It does not identify the exact rear-lens focal length or rule out image cropping. The image center is $(W/2, H/2) = (2856, 2142)$, while the principal-point offset is

$$(c_x - W/2, c_y - H/2) = (27.66418, -34.79706) \text{ px.}$$

Its Euclidean length is 44.45382 px, approximately 0.7783% of the image width and 1.0377% of its height. The principal point is therefore near, but not identical to, the image center.

8.2 Distortion and Error

In the OpenCV order, the distortion vector is

$$(0.20245123, -0.29420837, -0.00326679, 0.00308671, -1.04554325).$$

The radial and tangential correction terms are nonzero in this model. Their signs and magnitudes alone do not prove correction quality; the corrected image in Figure 2 should be examined together with checkerboard edges in the original image.

The three error values are OpenCV RMS 2.06424401 px, mean pointwise reprojection error 1.65696931 px, and mean per-image RMS 1.90750642 px. They differ because the first aggregates squared point distances globally, the second averages point distances, and the third averages per-image RMS values. The per-image RMS range is 0.850436–3.625463 px; views with larger errors should be inspected if a refined acquisition is needed.

At the current resolution, the OpenCV RMS is approximately 0.03614% of the image width and 0.04818% of its height. Thus the error is small relative to the image dimensions, but a high-resolution image also makes a fixed pixel error appear smaller in relative terms. This metric is not an independent validation and should not be compared directly across resolutions without accounting for scale and acquisition conditions.

8.3 Data Quality and Pose Coverage

All 18 candidate JPEG files passed reading, 8×5 corner detection, and the resolution check. The failure count is zero, and every image contributes 40 corners. The run therefore passes the program-level data-integrity checks. A separate pose log is not available in the current report, so the successful detection count alone cannot prove uniform coverage of the image center, edges, distances, and tilt angles. The original acquisition should be reviewed, especially for views with larger per-image RMS.

Because `square_size=3.0` is in cm, the rotation vectors are dimensionless and the camera matrix and distortion coefficients remain pixel-coordinate quantities, while the translation vectors are measured in cm.

Table 4: Analysis summary based on the measured result

Check	Evidence	Cautious interpretation
Focal-length consistency	$\rho_f = 0.0003511422$	The two estimates are close
Principal point	Offset 44.45382 px	Small displacement from the center
Distortion	Complete five-coefficient vector	Interpret with corrected image
Error metrics	2.06424401/1.65696931/1.90750642 px	Three definitions are recorded
Data integrity	18 candidates, 18 valid, 0 failed	Program checks passed
Pose coverage	No independent pose log	Coverage cannot be claimed complete
Physical scale	<code>square_size=3.0</code> cm	Translation vectors are in cm

Overall, the data support a reproducible camera matrix and distortion model for the current image-coordinate setup, with translation vectors in centimeters. The exact rear-lens focal length and the detailed pose-coverage record remain undocumented, so the result should not be presented as a universal accuracy guarantee for every lens mode or scene.

9 Conclusion

This experiment completed OpenCV camera calibration using 18 real JPEG images from the rear camera of an iPhone 15 Pro. The calibration used an 8×5 checkerboard-inner-corner configuration and a 3 cm square size. The program produced the camera matrix, five distortion coefficients, 18 rotation vectors, and 18 translation vectors, and recorded OpenCV RMS, mean pointwise reprojection error, and mean per-image RMS. All 18 candidate images were valid; no image-read, corner-detection, or resolution failures occurred.

The run used `square_size=3.0`, so translation vectors are expressed in cm, while the camera matrix and pixel-coordinate distortion results describe the present image model. JPEG EXIF and the acquisition record identify the Apple iPhone 15 Pro rear camera. The exact rear-lens focal length was not recorded and is not inferred here. A detailed pose-coverage log should still be added before submission, but it does not affect the numerical readback of the official NPZ result.